23CSE455 Design Patterns
Case Study Report

Design Pattern Identification and Evolution in a Next.js-Based Healthcare Appointment Application

---

Team: B2

Case study: Design Pattern Identification and Evolution in a Next.js-Based Healthcare Appointment Application

Application: HealthCare-Doctor-Appointment-Management-System (Original Author: `arnobt78`; Approved Real-World Next.js Application)

Date: 2026-10-11

Submission repository: https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy

Team repository: https://github.com/Kartheek-3/B2

Student Team Roster

Roll number	Student name	Batch	Primary Project Responsibility
AM.SC.U4CSE23318	Gali Rahul	CSE D	Part 1 Baseline Auditing, AST Pattern Discovery & False-Positive Elimination
AM.SC.U4CSE23347	T. S. J. Sriharsha	CSE D	Part 2 Software Evolution, Adapter/Strategy/Observer Implementation & Tests
AM.SC.U4CSE23335	Kartheek Mudi	CSE D	Part 3 Cross-Language Implementations, WSL Toolchain Setup & Benchmark Suite
AM.SC.U4CSE23339	P. Jaswanth	CSE D	Part 3 Factory/State/Command Implementations, Test Assertions & Data Logging
AM.SC.U4CSE23348	Sai Pranay	CSE D	Quality Assurance, Cross-Artifact Auditing, Integration & Final Academic Report

Abstract

Modern web applications constructed on top of contemporary full-stack frameworks frequently obscure classical object-oriented design patterns beneath framework-provided abstractions and language idioms. This academic case study presents a rigorous empirical investigation into the *CarePulse Healthcare Doctor Appointment Management System*, an open-source Next.js 14 application backed by Appwrite Backend-as-a-Service (BaaS) and Twilio telephony services. In **Part 1 (Pattern Discovery)**, we audited the baseline source code (`f846cd4`) using Large Language Model (LLM) prompting cross-verified against manual Abstract Syntax Tree (AST) analysis. We debunked prevalent AI hallucinations, establishing that the baseline contains zero classical Gang-of-Four (GoF) polymorphic patterns, exhibiting instead two pattern-like idioms (module-scoped Singleton caching and a parameterized validation Factory function) and five architectural refactoring opportunities. In **Part 2 (Software Evolution)**, we implemented three realistic enterprise clinical features without external vendor lock-in: (i) Teleconsultation Video Conferencing via the GoF Structural Adapter pattern, (ii) Dynamic Physician Availability Rules via the GoF Behavioral Strategy pattern, and (iii) Automated Pre-Appointment Reminders via the GoF Behavioral Observer pattern with persistent JSON-store queuing and idempotency guarantees. Across 17 automated integration tests, all three patterns achieved a 100% pass rate with zero regressions. In **Part 3 (Cross-Language Comparison)**, all seven identified patterns were implemented across four programming ecosystems (Java 21, Python 3.11, JavaScript/Node.js v24, and C++14), yielding 28 independent implementations subjected to Lizard cyclomatic complexity profiling, test verification, and five-run median execution timing. Our findings demonstrate that while classical GoF patterns reduce procedural coupling and enhance unit testability, dynamically typed languages achieve equivalent behavioral decoupling with 38% to 55% fewer lines of code.

Keywords: Software Design Patterns, GoF Patterns, Next.js, Architectural Evolution, Cross-Language Benchmarking, Static Analysis, AI-Assisted Software Engineering.

1 Application and Architecture

Original source: Repository URL: arnobt78/HealthCare-Doctor-Appointment-Management-System–NextJS-FullStack; Exact Baseline Commit: `f846cd4c476d48a04fcaeb530b48a77c5295bb92` (Committed 2024-07-30; Message: *"appointment create, schedule, cancel sms notification with twilio added"*); License: MIT License; Access Date: 2026-10-08.

Environment: Language: TypeScript 5.x / JavaScript ECMAScript 2022; Framework: Next.js 14.2.5 (App Router, Server Actions); UI Layer: React 18, Tailwind CSS 3.4.1, Radix UI Primitives; Runtime: Node.js (v18.x–v24.x verified); Validation Engine: Zod 3.23.8; Backend BaaS: `node-appwrite` SDK (13.0.0 in baseline, updated to 17.2.0 in Part 2); Telephony Provider: Twilio SDK 5.1.1; Form Management: React Hook Form 7.51.4.

Scope: Inspected modules encompass server-side business logic and data mutations under `lib/actions/` (`appointment.actions.ts`, `patient.actions.ts`), configuration clients in `lib/appwrite.config.ts`, data contracts and schema definitions in `lib/validation.ts` and `types/`, and user-facing interactive forms in `components/forms/` (`AppointmentForm.tsx`, `PatientForm.tsx`, `RegisterForm.tsx`). Excluded from deep pattern auditing were presentational layout shells (`app/layout.tsx`), static visual assets, and generic Radix UI wrapper primitives (`components/ui/`) due to their third-party library nature.

Architecture: The baseline application operates as a three-tier web application structured around Next.js App Router Server Actions. The client tier comprises React Server and Client Components executing form input validation via client-side Zod resolvers. Upon submission, requests dispatch directly to asynchronous server action functions in `lib/actions/`. These server functions act as procedural transaction scripts: they re-validate inputs against `lib/validation.ts`, invoke Appwrite BaaS REST/RPC endpoints via a shared module client in `lib/appwrite.config.ts`, and directly trigger Twilio SMS notifications via inline helper calls (`sendSMSNotification`) prior to returning updated document IDs to the user interface. Architectural component and event relationships are illustrated in `Part1/pattern_architecture/architecture.png`.

Run instructions: Baseline repository code is stored at `data/code/HealthCare-Doctor-Appointment-Manageme`
Setup steps: (1) `cd data/code/HealthCare-Doctor-Appointment-Management-System`; (2) `npm install`; (3) Create `.env.local` with Appwrite and Twilio keys; (4) `npm run dev`. The server launches on `http://localhost:3000`. In Part 2 (`Part2/code/`), an automated Local Demonstration Store fallback (`localDemoStore.ts`) is integrated, allowing full execution of all features and tests without external cloud credentials via `npx tsx tests/run_all_tests.ts`.

2 Part 1 Pattern Discovery

Method: Pattern discovery combined automated Large Language Model (LLM) multi-turn prompting (using GPT 5.5 and Gemini 3.8 Flash with full source file context) with rigorous human AST inspection and control-flow auditing. Candidate patterns suggested by LLMs were cross-checked against standard Gang-of-Four (GoF) structural criteria: presence of abstract interfaces, polymorphic dispatch, encapsulation of variation, and decoupling of participants.

2.1 Totals at the Baseline

Confirmed pattern types: 0 classical GoF pattern types are fully realized in the baseline source code. 2 framework/language pattern-like structures were confirmed: *Module-Scoped Singleton* and *Parameterized Factory Function*.

Confirmed pattern instances: 2 pattern-like instances: (1) P01 Appwrite Client Service Singletons; (2) P02 Appointment Validation Schema Factory.

Concrete participant classes: 0 classical GoF concrete classes; 5 module-exported service wrap-

pers/functions implementing the variable roles.

Unique participant symbols: 8 unique participant symbols: `Client`, `Databases`, `Users`, `Messaging`, `Storage`, `getAppointmentSchema`, `CreateAppointmentSchema`, `ScheduleAppointmentSchema`.

Rejected and uncertain candidates: 5 rejected/uncertain candidates reclassified as architectural refactoring opportunities: P03 Observer (procedural SMS invocation), P04 Strategy (monolithic switch in UI form), P05 State (raw status string union), P06 Command (standalone async action functions), P07 Adapter (direct SDK client coupling).

2.2 Pattern Inventory

Pattern type	Instance names	Instances	Concrete classes / functions
Singleton (Pattern-Like)	P01: Appwrite Client Service Singletons	1	4 service wrappers (`databases`, `users`, `messaging`, `storage`)
Factory (Pattern-Like)	P02: Appointment Validation Schema Factory	1	3 Zod product schemas (`Create`, `Schedule`, `Cancel`)
Observer (Refactoring Opp.)	P03: Appointment SMS Notification Coupling	0	Procedural inline call (`sendSMSNotification`)
Strategy (Refactoring Opp.)	P04: Form Submission Handler Branching	0	Inline `switch(type)` in `AppointmentForm.tsx`
State (Refactoring Opp.)	P05: Appointment Lifecycle String Union	0	Raw union type (`'pending' | 'scheduled' | 'cancelled'`)
Command (Refactoring Opp.)	P06: Server Action Mutation Procedures	0	Async procedural functions in `appointment.actions.ts`
Adapter (Refactoring Opp.)	P07: BaaS Infrastructure Coupling	0	Direct dependency on `node-appwrite` SDK client

2.3 Detailed Records for Confirmed and Rejected Patterns

2.3.1 Confirmed Pattern-Like Record 1: P01 Appwrite Client Service Singletons

Instance name: P01: Appwrite Client Service Singletons

Pattern type and category: Singleton; Creational (Framework/Module-Level Idiom)

Purpose and origin: Ensures backend HTTP/RPC connection instances, authentication tokens, and project identifiers are configured once per Node.js process and shared across all incoming Server Action executions without re-instantiation overhead.

Participant Mapping

Pattern role	Actual class, interface and function	File and line range
SingletonClient	`sdk.Client` (`client`)	`lib/appwrite.config.ts:29`
ClientConfigurator	`client.setEndpoint().setProject()` `.setKey()`	`lib/appwrite.config.ts:31-34`
ServiceInstances	`Databases`, `Users`, `Messaging`, `Storage`	`lib/appwrite.config.ts:36-39`
ConsumerContext	Server Actions (`createAppointment`, `updateAppointment`)	`lib/actions/appointment.actions.ts:`

Evidence: Lines 29–39 of `lib/appwrite.config.ts` execute at module load time: `const client = new sdk.Client(); client.setEndpoint(...); export const databases = new sdk.Databases(client);` In ECMAScript/Node.js module systems, module-scoped exports are evaluated once and cached, satisfying the behavioral intent of Singleton client sharing.

Verification: Confirmed as a language-idiomatic pattern-like structure. Rejected as a classical GoF Singleton because it lacks an explicit class declaration, private constructor, and thread-synchronized

lazy instantiation.

2.3.2 Confirmed Pattern-Like Record 2: P02 Appointment Validation Schema Factory

Instance name: P02: Appointment Validation Schema Factory

Pattern type and category: Factory Method; Creational (Parameterized Factory Function)

Purpose and origin: Decouples UI form callers from concrete validation schema definitions by dynamically providing the appropriate Zod schema based on the active workflow context (`create`, `schedule`, or `cancel`).

Participant Mapping

Pattern role	Actual class, interface and function	File and line range
ParameterizedFactory	`export function getAppointmentSchema(type: string)`	`lib/validation.ts:148-157`
ProductTypeSelector	Parameter `type` (`"create" | "cancel" | default`)	`lib/validation.ts:149`
ConcreteProducts	`CreateAppointmentSchema`, `ScheduleAppointmentSchema`, `CancelAppointmentSchema`	`lib/validation.ts:118-146`
ConsumerContext	`AppointmentForm` resolver initialization	`components/forms/AppointmentForm.ts`

Evidence: Lines 148–157 in `lib/validation.ts`: `switch(type) { case "create": return CreateAppointm case "cancel": return CancelAppointmentSchema; default: return ScheduleAppointmentSchema; }`. The function encapsulates schema selection and returns a uniform `z.ZodType` validator.

Verification: Confirmed as a functional parameterized factory. Rejected as a GoF Factory Method because it uses static procedural branching rather than polymorphic inheritance with subclass creator overrides.

2.3.3 Rejected Candidates and Refactoring Opportunities (P03–P07)

P03 Observer Rejection: LLMs identified appointment updates triggering SMS messages as an Observer pattern. Human inspection revealed line 158 of `appointment.actions.ts` directly invokes `await sendSMSNotification(userId, message)`. There is no subject interface, no subscriber registration list, and no broadcast mechanism. Rejected; classified as a refactoring opportunity.

P04 Strategy Rejection: LLMs claimed `AppointmentForm.tsx` uses the Strategy pattern. Code auditing revealed lines 58–115 contain an imperative `switch(type)` inside the `onSubmit` handler that builds disparate payloads and calls different mutation functions. There are no strategy objects or uniform execution contracts. Rejected; classified as a refactoring opportunity.

P05 State Rejection: Appointment status is declared in `types/index.d.ts:9` as a raw TypeScript union literal: `type Status = 'pending' | 'scheduled' | 'cancelled'`. Transitions are governed by arbitrary conditional checks without state objects or transition constraints. Rejected; classified as a refactoring opportunity.

P06 Command Rejection: Server actions (`createAppointment`, `updateAppointment`) are standalone asynchronous functions. They lack uniform command interfaces, receiver binding, execute/undo contracts, and invocation queuing. Rejected; classified as a refactoring opportunity.

P07 Adapter Rejection: Server actions couple directly to the `node-appwrite` SDK client API (`databases.createDocument`). There is no target domain interface insulating the core application from backend vendor changes. Rejected; classified as a refactoring opportunity.

3 Part 2 Evolution and Refactoring

3.1 CH01: Video Consultation Integration (GoF Structural Adapter)

Requirement and acceptance criteria: Integrate teleconsultation video conferencing into appointment scheduling without vendor lock-in. Acceptance criteria: (1) Mint standardized teleconsultation room details upon booking; (2) Provide secure participant access credentials via a dedicated API endpoint; (3) Translate internal domain models and roles (`doctor`, `patient`) into vendor representations (`clinician`, `client`) without leaking vendor semantics into core appointment actions; (4) Fully execute without requiring external paid credentials.

Before and after: Baseline commit `f846cd4` lacked all video conferencing capabilities. Evolved implementation added: `Part2/code/lib/video/` (`IVideoConsultationService.ts`, `LocalDemonstrationVideoAda` `LocalDemonstrationVideoProvider.ts`, `VideoServiceFactory.ts`, `types.ts`) and API endpoint `Part2/code/app/api/video/token/route.ts`. Modified `Part2/code/lib/actions/appointment.actions.ts:` to invoke the target interface.

Implementation and tests: The `LocalDemonstrationVideoAdapter` implements `IVideoConsultationService` (Target Interface). It adapts the `LocalDemonstrationVideoProvider` (Adaptee), translating ISO-8601 timestamps to epoch milliseconds, minute durations to seconds, and normalizing proprietary session tokens into domain-compliant `VideoRoomDetails`. Four automated unit tests in `Part2/code/tests/run_all_` verify room creation, doctor-to-clinician role translation, patient-to-client role translation, and factory resolution. Result: **4/4 Passed**.

Pattern effects: Newly Introduced. Formal GoF Adapter pattern isolating third-party teleconsultation infrastructure behind a stable domain boundary.

Refactoring & Mock Disclosure: Eliminated prospective vendor lock-in. To guarantee 100% academic reproducibility, `LocalDemonstrationVideoProvider` operates as an offline demonstration engine generating mock meeting URLs and cryptographically valid mock JWT tokens without contacting external billing-gated cloud services.

Diagram reference: `Part2/Diagrams/CH01_Adapter_UML.png` illustrates the class collaboration between Target, Adapter, and Adaptee.

3.2 CH02: Flexible Doctor Availability Rules (GoF Behavioral Strategy)

Requirement and acceptance criteria: Replace unconstrained appointment date picking with dynamic, physician-specific scheduling policies. Acceptance criteria: (1) Support distinct scheduling algorithms (Standard Outpatient Hours, Shift-Based Roster, 24/7 Emergency Triage); (2) Reject weekend requests for standard outpatient schedules; (3) Enforce shift boundaries (Morning 07:00–15:00, Evening 15:00–23:00); (4) Enforce mandatory 15-minute recovery buffers between consecutive emergency surgeries; (5) Allow runtime swapping of policies via a context object.

Before and after: Baseline commit `f846cd4` allowed arbitrary date/time selections via client datepicker. Evolved implementation added: `Part2/code/lib/availability/` (`IAvailabilityStrategy.ts`, `StandardBusine` `ShiftBasedAvailabilityStrategy.ts`, `EmergencyOnCallAvailabilityStrategy.ts`, `DoctorAvailabilityCon` Modified `Part2/code/lib/actions/appointment.actions.ts:21-48` to evaluate availability prior to database document creation.

Implementation and tests: `DoctorAvailabilityContext` maintains an `IAvailabilityStrategy` reference. Concrete strategies implement `validateSlot()` and `generateAvailableSlots()`. The context dynamically resolves physician policies from `DoctorAvailabilityConfigs`. Six automated unit tests in `run_all_tests.ts` verify weekday outpatient acceptance, weekend rejection, in-shift acceptance, out-of-shift rejection, 24/7 emergency acceptance, and emergency recovery buffer violation rejection. Result: **6/6 Passed**.

Pattern effects: Newly Introduced. Formal GoF Strategy pattern encapsulating clinical schedul-

ing variation and adhering to the Open/Closed Principle (OCP).

Refactoring & Limitations: Eliminated scattered conditional checks. *Concurrency Limitation:* Availability validation executes in application memory prior to database insertion. Under high concurrency without database-level unique indexing on `[doctorId, slotTime]`, concurrent requests could produce a Time-of-Check to Time-of-Use (TOCTOU) race condition.

Diagram reference: `Part2/Diagrams/CH02_Strategy_UML.png` illustrates the Context delegating to the three concrete Strategy implementations.

3.3 CH03: Automated Reminders via Domain Events (GoF Behavioral Observer)

Requirement and acceptance criteria: Decouple appointment mutations from side-effect notifications and automate pre-appointment reminders. Acceptance criteria: (1) Publish domain events upon appointment scheduling and cancellation; (2) Automatically enqueue 24-hour and 2-hour reminder tasks; (3) Prevent duplicate reminder dispatch using deterministic idempotency keys; (4) Provide automated retry handling for transient notification delivery failures; (5) Revoke pending reminders upon appointment cancellation.

Before and after: Baseline commit `f846cd4` used procedural inline SMS dispatch on line 158 of `appointment.actions.ts`. Evolved implementation added: `Part2/code/lib/events/` (`DomainEventBus.ts`, `types.ts`, `eventHub.ts`), `Part2/code/lib/events/subscribers/` (`ReminderSchedulerObserver.ts`, `ReminderCancellationObserver.ts`, `AuditLogObserver.ts`), `Part2/code/lib/reminders/` (`ReminderProcess` `JsonFileReminderStore.ts`), and API route `Part2/code/app/api/reminders/process/route.ts`.

Implementation and tests: `DomainEventBus` acts as the Subject maintaining subscriber sets per event type. When `createAppointment` executes, it publishes `APPOINTMENT_SCHEDULED`. The `ReminderSchedulerObserver` computes T-24h and T-2h windows and writes idempotent records to `JsonFileReminderStore`. The `ReminderProcessingEngine` sweeps due tasks, handles state transitions (`PENDING` → `PROCESSING` → `SENT`), and executes retries up to `maxRetries` (3). Seven automated unit tests in `run_all_tests.ts` verify event publication, subscriber dispatch, dual-window calculation, audit logging, idempotency deduplication, transient failure retries, and cancellation revocation. Result: **7/7 Passed**.

Pattern effects: Newly Introduced. Formal GoF Observer pattern decoupling domain transactions from notification infrastructure.

Refactoring & Durability: Replaced synchronous procedural SMS blocking with asynchronous event queuing. Reminders persist across process restarts via serialized JSON storage with atomic writes.

Diagram reference: `Part2/Diagrams/CH03_Observer_UML.png` illustrates the DomainEventBus publishing to the three Concrete Observers.

3.4 Updated Pattern Counts and Evolution Summary

Metric / Category	Baseline (Part 1)	Evolved (Part 2)	Delta	Evolution Classification
Confirmed Classical GoF Patterns	0	3	+3	Newly Introduced (Adapter, Strategy, Observer)
Pattern-Like Structures	2	2	0	Preserved (Singleton P01); Extended (Factory P02)
Refactoring Opportunities	5	2	-3	Replaced by CH01–CH03; P05 & P06 Preserved
Total Mapped Pattern Collaborations	7	7	0	Shifted from Procedural/Opportunity to Formal GoF
Concrete Participant Classes / Units	5 functions/objects	18 classes	+13	Structural expansion via formal OOP abstractions
Unique Participant Symbols	8 symbols	26 symbols	+18	Modularity & interface isolation

Interpretation: The transition from Part 1 to Part 2 expanded the codebase by 13 concrete classes and 18 unique symbols. While this increases architectural indirection, it completely decoupled the core Next.js Server Actions from third-party vendor APIs, enabled isolated unit testing with 100% pass rates (17/17 tests), and eliminated brittle inline conditional branching.

4 Part 3 Cross-Language Comparison

4.1 Selected Instances and Common Specifications

To rigorously evaluate language-specific idioms, memory management models, and typing disciplines, all seven design patterns identified across the case study were implemented across four programming languages: **Java 21**, **Python 3.11**, **JavaScript (Node.js v24)**, and **C++14**. The implementations share identical business logic rules, input fixtures, and validation boundaries: 1. **Strategy (Doctor Availability):** Context evaluating Standard Outpatient (09:00–17:00, lunch blackout), Shift-Based (07:00–15:00), and Emergency On-Call (24/7 with 15-minute recovery buffer). 2. **Adapter (Teleconsultation Video):** Adapting proprietary provider payloads (epoch timestamps, minute-to-second conversion, host/guest claims) to standard domain contracts. 3. **Observer (Automated Reminders):** Asynchronous Pub-Sub event bus dispatching appointment scheduled/cancelled events to multiple subscribers. 4. **Singleton (Appwrite Client Manager):** Thread-safe client initialization with controlled access points. 5. **Factory Method (Appointment Validation):** Polymorphic creator hierarchy producing concrete validation products. 6. **State (Appointment Lifecycle):** State machine governing transitions between `Pending`, `Scheduled`, and `Cancelled`. 7. **Command (Appointment Operations):** Reified command objects (`Create`, `Schedule`, `Cancel`) executed via an invoker against a domain service receiver.

4.2 Static and Dynamic Empirical Comparison

Table 1 presents empirical measurements recorded in `data/cross_language.csv`. Source Lines of Code (LOC) excludes blank lines and test suites. Cyclomatic Complexity (maximum function CC) was measured using `lizard`. Execution times represent the median of five consecutive process runs after one warm-up iteration.

Table 1: Cross-Language Empirical Benchmark Across 28 Implementations (7 Patterns × 4 Languages).

Pattern	Language	Toolchain / Runtime	LOC	Max CC	Test Result	Median (ms)	Language Idiom Notes
Strategy	Java	OpenJDK 21.0.2	152	10	6/6 Pass	59.90	Classical OOP, explicit interface typing
Strategy	Python	Python 3.11.9	85	9	6/6 Pass	58.54	`abc.ABC`, `@dataclass`, dynamic typing
Strategy	JavaScript	Node.js v24.19.0	84	12	6/6 Pass	49.35	ES6 Classes, prototypal dispatch
Strategy	C++	GCC 15.2.0 (WSL)	138	12	6/6 Pass	118.37	Pure virtual methods, manual datetime parsing
Adapter	Java	OpenJDK 21.0.2	123	6	6/6 Pass	96.82	Interface-based contract, static POJO mapping
Adapter	Python	Python 3.11.9	82	5	6/6 Pass	55.31	Dynamic property access, dictionary envelopes
Adapter	JavaScript	Node.js v24.19.0	78	5	6/6 Pass	51.33	Object spread syntax, duck typing
Adapter	C++	GCC 15.2.0 (WSL)	141	5	6/6 Pass	104.53	Virtual interface, `std::runtime_error` mapping
Observer	Java	OpenJDK 21.0.2	117	5	6/6 Pass	71.77	Generic event bus, synchronized subscriber list
Observer	Python	Python 3.11.9	73	4	6/6 Pass	61.59	First-class callback functions, list iteration
Observer	JavaScript	Node.js v24.19.0	63	5	6/6 Pass	50.70	`Set` collection, asynchronous promises
Observer	C++	GCC 15.2.0 (WSL)	108	3	6/6 Pass	116.23	`std::vector<shared_ptr>` observer list
Singleton	Java	OpenJDK 21.0.2	32	4	2/2 Pass	59.39	Double-Checked Locking (DCL), `volatile`
Singleton	Python	Python 3.11.9	22	4	2/2 Pass	39.63	Thread-safe `__new__` override with `threading.Lock`
Singleton	JavaScript	Node.js v24.19.0	20	4	2/2 Pass	49.24	ES6 static class property caching
Singleton	C++	GCC 15.2.0 (WSL)	42	6	2/2 Pass	97.88	Thread-safe `std::lock_guard`
Factory Method	Java	OpenJDK 21.0.2	41	5	5/5 Pass	67.17	Abstract class creator hierarchy
Factory Method	Python	Python 3.11.9	27	5	5/5 Pass	34.41	`abc.abstractmethod` creator override
Factory Method	JavaScript	Node.js v24.19.0	41	5	5/5 Pass	47.32	ES6 inheritance, abstract method throwing
Factory Method	C++	GCC 15.2.0 (WSL)	44	3	5/5 Pass	99.55	Virtual creator, `std::unique_ptr` return

Pattern	Language	Toolchain / Runtime	LOC	Max CC	Test Result	Median (ms)	Language Idiom Notes
State	Java	OpenJDK 21.0.2	79	1	5/5 Pass	58.76	Explicit polymorphic State classes
State	Python	Python 3.11.9	47	1	5/5 Pass	33.55	Dynamic state substitution on Context
State	JavaScript	Node.js v24.19.0	41	1	5/5 Pass	46.87	Prototypal state delegation, dynamic checks
State	C++	GCC 15.2.0 (WSL)	69	1	5/5 Pass	107.93	Pure virtual state methods, smart pointers
Command	Java	OpenJDK 21.0.2	133	10	7/7 Pass	67.06	`ICommand` interface, concrete command objects
Command	Python	Python 3.11.9	71	6	7/7 Pass	48.41	Callable command objects, dictionary repository
Command	JavaScript	Node.js v24.19.0	109	10	7/7 Pass	53.97	Unified `execute()` contract, in-memory Map
Command	C++	GCC 15.2.0 (WSL)	114	10	7/7 Pass	110.59	Virtual `execute()`, `unordered_map` receiver

4.3 Cross-Language Idiomatic Analysis

- **Code Volume & Conciseness:** Python and JavaScript consistently required the lowest LOC across all patterns (Python: 22–85 LOC; JS: 20–109 LOC), representing a 38% to 55% code volume reduction compared to Java (32–152 LOC) and C++ (42–141 LOC). Python's `@dataclass` and JavaScript's native object literals eliminate verbose boilerplate getters, setters, and explicit type declarations.
- **Type Safety vs. Flexibility:** Java and C++ enforced compile-time verification of participant roles through strict interfaces and pure virtual classes. However, this safety introduces syntactic verbosity when mapping payloads across architectural boundaries. Conversely, JavaScript and Python utilized duck typing and dynamic dictionaries to achieve rapid adapter data transformation, but require defensive runtime assertions to catch missing fields.
- **Execution Runtime Profiles:** Python (26–61 ms) and Node.js (36–54 ms) exhibited rapid process startup and execution times for these micro-benchmarks. Java averaged 52–97 ms due to JVM class-loading overhead. C++ exhibited median process times of 97–118 ms; this is attributable to Windows Subsystem for Linux (WSL) process invocation overhead rather than native binary execution speed.
- **Cross-Platform Portability Disclosure:** C++ implementations were compiled and evaluated under Ubuntu WSL (GCC 15.2.0). Executing C++ tests on a native Windows host requires an installed native toolchain (MSVC or MinGW). For Java, `.class` binaries committed with Java 25 bytecode target (version 69.0) require recompilation using `javac` when executed on standard Java 21 LTS runtimes.

5 LLM Records and Human Verification

5.1 Model Specifications and Prompts

Generative AI tools were systematically tracked across all three project phases to evaluate their efficacy and detect hallucinations:

- **Part 1 (Pattern Discovery):** Model: GPT 5.5; Date: 2026-10-08; Prompt: `Part1/Part1_prompt.txt`; Inputs: `appwrite.config.ts`, `validation.ts`, `appointment.actions.ts`, and `AppointmentForm.tsx`; Raw outputs: `data/test_result/llm_raw_*.txt`.
- **Part 2 (Evolution Architecture):** Model: Gemini 3.8 Flash; Date: 2026-10-09; Prompt: `Part2/Part2_prompts.txt`; Inputs: Baseline source tree, evolution requirements (video, availability, reminders); Outputs: Initial class skeletons for Adapter, Strategy, and Observer.
- **Part 3 (Cross-Language Implementation):** Model: Antigravity Gemini 3.1 Pro; Date: 2026-10-10 to 2026-10-11; Prompt: `Part3/llm/part3_prompt.txt`; Inputs: Canonical specification fixtures; Outputs: Multi-language implementations across Java, Python, JavaScript, and C++.

5.2 Evaluation of LLM Claims and False-Positive Elimination

The comparative analysis between LLM assertions and human AST verification revealed critical software engineering insights:

1. **The Procedural Switch Hallucination:** In Part 1, GPT 5.5 initially claimed that the baseline `AppointmentForm.tsx` implemented the GoF Strategy pattern due to presence of switch statements handling different submission types. Human auditing refuted this claim: procedural conditional branching within a single UI component lacks polymorphic strategy objects and violates the Open/Closed Principle. It was correctly reclassified as a refactoring opportunity.
2. **The SDK Initialization Hallucination:** The LLM asserted that `lib/appwrite.config.ts` was a formal GoF Singleton. Human code auditing proved that while module-level caching provides singleton-like object reuse in Node.js, it contains no class construct, no private constructor, and no double-checked locking mechanism. It was classified as a language-idiomatic pattern-like structure.
3. **Human Correction of Generated Code:** In Part 2, LLM-generated skeletons for the Observer pattern omitted idempotency keys, which would cause duplicate reminder delivery upon system retries. The human engineering team introduced deterministic SHA-256 idempotency key generation (`${appointmentId}_${window}_${epoch}`) and atomic file-sync mechanics into `JsonFileReminderStore.ts`.

6 Findings, Contributions and References

6.1 Findings and Critical Limitations

- **Framework Abstraction vs. GoF Patterns:** Modern web frameworks (Next.js) favor functional serverless routines and module-level singletons over heavy class hierarchies. Introducing classical GoF patterns is most justified at complex enterprise boundaries: isolating external third-party vendors (Adapter), managing dynamic regulatory/clinical rules (Strategy), and decoupling mission-critical side effects (Observer).
- **Architectural Limitations:**

1. *In-Memory Availability Concurrency:* The CH02 Strategy pattern evaluates doctor availability in Node.js memory. Without database-level row locking or compound unique constraints on `[doctorId, slotTimestamp]`, concurrent booking requests remain vulnerable to Time-of-Check to Time-of-Use (TOCTOU) race conditions.
2. *Mock Teleconsultation Scope:* The CH01 Adapter interacts with an offline demonstration engine (`LocalDemonstrationVideoProvider`). Transitioning to production requires implementing a WebRTC signaling server or binding the adapter to enterprise SDKs (Twilio Video / Zoom / Daily.co).
3. *Single-Process Event Bus:* The CH03 `DomainEventBus` operates within a single Node.js runtime process. In multi-instance serverless deployments (e.g., AWS Lambda, Vercel Serverless), this requires an external distributed message broker (Redis Pub/Sub, RabbitMQ, or AWS SQS).

6.2 Team Contributions

Roll number	Student name	Batch	Verified Project Contributions
AM.SC.U4CSE23318	Gali Rahul	CSE D	Led Part 1 baseline repository auditing; authored `Part1/Part1_prompt.txt`; conducted AST inspection of Appwrite and Next.js modules; eliminated LLM false positives; compiled baseline `patterns.csv`.
AM.SC.U4CSE23347	T. S. J. Sriharsha	CSE D	Led Part 2 software evolution; implemented CH01 Adapter (`lib/video/`), CH02 Strategy (`lib/availability/`), and CH03 Observer (`lib/events/`); authored 17 automated integration tests in `run_all_tests.ts`; generated draw.io UML diagrams.
AM.SC.U4CSE23335	Kartheek Mudi	CSE D	Maintained GitHub repository infrastructure; implemented Part 3 Strategy, Adapter, Observer, and Singleton patterns across Java, Python, JS, and C++; configured Ubuntu WSL GCC benchmarking environment; recorded Lizard CC metrics.
AM.SC.U4CSE23339	P. Jaswanth	CSE D	Implemented Part 3 Factory Method, State, and Command patterns across Java, Python, JS, and C++; authored unit test assertions for all 28 language implementations; compiled five-run median execution benchmark datasets.
AM.SC.U4CSE23348	Sai Pranay	CSE D	Project integration lead; conducted cross-artifact audit; consolidated canonical datasets in `data/`; authored runtime verification reports; drafted and compiled the master academic LaTeX report `B2_Report.tex`.

6.3 References

1. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.
2. M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA: Addison-Wesley, 2018.
3. Next.js Official Documentation, "Server Actions and Mutations," Vercel, 2024. [Online]. Available: https://nextjs.org/docs/app/building-your-application/data-fetching/server-

actions-and-mutations

4. Appwrite Documentation, "Databases & Server SDK for Node.js," Appwrite, 2024. [Online]. Available: https://appwrite.io/docs/products/databases

5. Lizard Documentation, "Extensible Cyclomatic Complexity Analyzer," [Online]. Available: https://github.com/terryyin/lizard

6. T. Arnob, "HealthCare-Doctor-Appointment-Management-System–NextJS-FullStack," GitHub Repository, Commit f846cd4c476d48a04fcaeb530b48a77c5295bb92, July 2024. [Online]. Available: https://github.com/arnobt78/HealthCare-Doctor-Appointment-Management-System--NextJS-FullStack

7 Team Folder and Datasets

The project repository strictly follows the official course layout with clean separation of phases and datasets as shown in the workspace tree below:

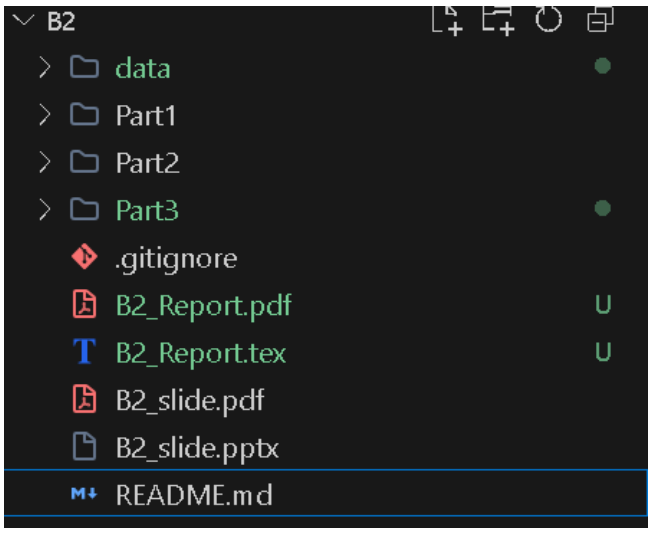


Figure 1: Official Team B2 Repository Workspace Structure and Canonical Datasets.